

Zaun — Design Doc & Build Plan

2026-09-22 · @Someone

Overview

Zaun is a C++17 command-line sandbox that runs untrusted, LLM-generated code on Linux with only the access that code needs. `zaun learn` traces a run and writes a least-privilege policy; `zaun run` enforces a policy with namespaces, Landlock, seccomp-bpf and cgroups v2. The MVP ships by 2026-11-01.

Problem. Coding agents now write code and run it as the user. A prompt-injected script can read `~/.ssh`, exfiltrate API keys from the environment, or edit `~/.bashrc`. Existing sandboxes (bubblewrap, nsjail, Docker) enforce policies well, but someone has to write the policy, so people grant broad access or skip sandboxing.

Zaun's contribution is the loop around enforcement: learn a policy from a real run, show it for review, enforce it, and report violations. The evaluation question is whether a policy learned from one LLM sample still works for other samples of the same task.

Goals

- Block every test in the escape suite (about 28 attacks) as an unprivileged user, after one-time setup.
- Add under 25 ms of startup latency and under 5% slowdown on a syscall-heavy workload (targets, confirmed by measurement in week 5).
- Policies learned from one run succeed on at least 90% of reruns with freshly generated code for the same task.
- Integrate with an agent in one call: `zaun_exec(code, policy)` from Python.

Non-goals

- Defending against kernel exploits. gVisor or Firecracker is the right tool when that is in scope.
- macOS or Windows support, GPUs, container images.
- Replacing Docker or running as a multi-tenant service.

Threat model

Zaun assumes the code inside the sandbox is fully hostile and the Linux kernel is trusted. The attacker has arbitrary code execution as the invoking user and may have been steered by prompt injection.

Trust boundary. The `zaun` supervisor process and the policy file are trusted. Everything after the target's `execve` is untrusted, including anything it spawns. A learned policy is only trusted after the user reviews it.

Asset	Example attacks	Primary defense
Secrets on disk	Read <code>~/.ssh/id_ed25519</code> , <code>~/.aws/credentials</code> , browser profiles	Mount namespace hides <code>\$HOME</code> ; Landlock read rules
Secrets in memory	Read API keys from environment, inherited file descriptors	Environment allowlist; <code>close_range</code> on <code>fds ≥ 3</code>
Persistence	Write <code>~/.bashrc</code> , crontab, systemd user units	Read-only binds; only the <code>workdir</code> and <code>/tmp</code> are writable
Network	Exfiltrate over TCP/DNS; talk to D-Bus or X11 via abstract Unix sockets	Empty network namespace (also scopes abstract sockets)
Other processes	<code>ptrace</code> , <code>kill(-1)</code> , read <code>/proc/<pid>/environ</code>	PID namespace; seccomp denies <code>ptrace</code> , <code>process_vm_*</code>
Host terminal	Inject keystrokes with the <code>TIOCSITI</code> ioctl	<code>setsid()</code> ; seccomp filters <code>ioctl</code> request codes
Host resources	Fork bomb, memory balloon, infinite loop, fill <code>/tmp</code>	cgroup <code>pids.max</code> , <code>memory.max</code> , <code>timeout</code> + <code>cgroup.kill</code> , <code>tmpfs</code> size
Privilege boundary	Setuid binaries, nested user namespaces, <code>io_uring</code> to bypass seccomp, <code>bpf</code> , <code>userfaultfd</code>	<code>NO_NEW_PRIVS</code> ; always-deny seccomp list; all capabilities dropped

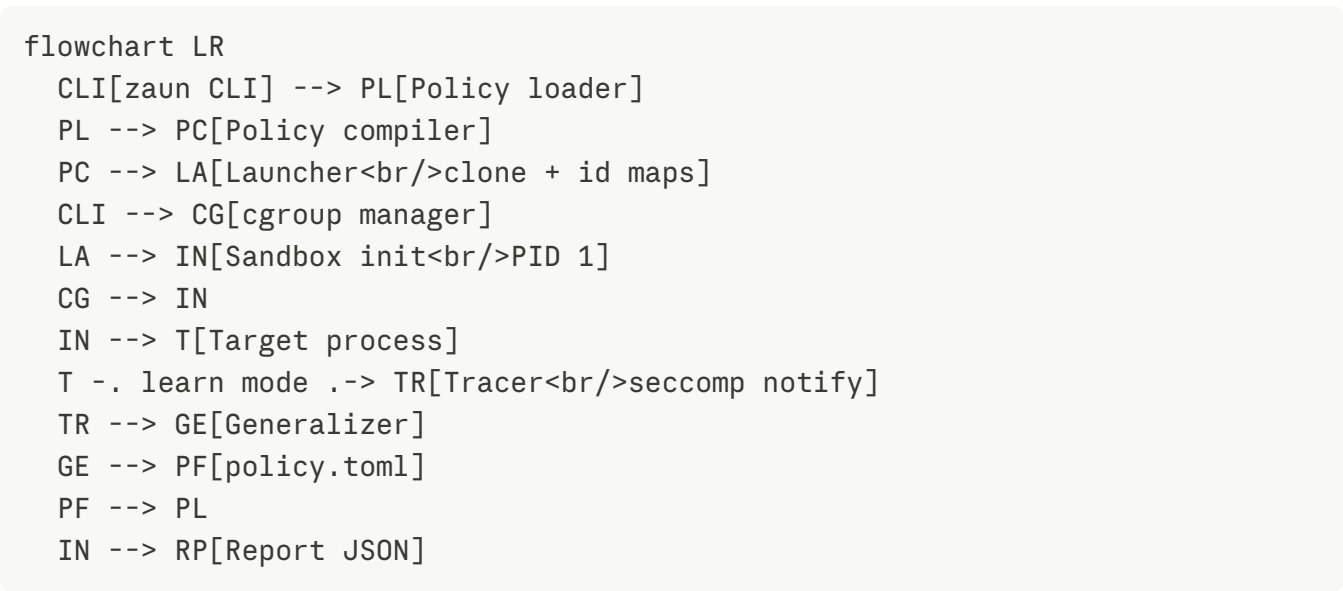
Out of scope

- Kernel vulnerabilities and container escapes that rely on them.

- Side channels (timing, Spectre) and covert channels through resources the policy allows.
- Malicious policy files, and attacks on the LLM itself.
- Data the policy deliberately exposes. If the user grants a directory, Zaun protects nothing inside it.

Architecture

Zaun is one binary with a trusted supervisor outside the sandbox and a small init process inside it. The policy compiler turns one policy file into four enforcement plans: mounts, Landlock rules, a seccomp filter and cgroup limits.



In `run` mode the loop stops at the target. In `learn` mode the tracer records accesses and the generalizer writes a policy that the user reviews and feeds back in.

Component	Responsibility	Key kernel APIs
CLI	Parse commands, load policy, print report	—
Policy loader	Parse and validate TOML; reject unknown keys	—
Policy compiler	Emit mount plan, Landlock ruleset, seccomp filter, limits	libseccomp, <code>landlock_*</code>

Component	Responsibility	Key kernel APIs
Launcher	Create namespaces, write uid/gid maps, sync with child	<code>clone</code> , <code>/proc/<pid>/uid_map</code> , <code>pipes</code>
cgroup manager	Create a child cgroup, set limits, kill on timeout	cgroup v2 files, <code>cgroup.kill</code>
Sandbox init	Build root, <code>pivot_root</code> , fork target, reap zombies, forward signals	<code>mount</code> , <code>pivot_root</code> , <code>waitpid</code>
Target setup	Drop privileges and lock down, then <code>execve</code>	<code>capset</code> , <code>prctl</code> , <code>landlock_restrict_self</code> , <code>seccomp</code>
Tracer	Receive syscall notifications, read path arguments, log	<code>SECCOMP_IOCTL_NOTIF_*</code> , <code>process_vm_readv</code>
Generalizer	Turn raw traces into minimal, readable rules	—
Reporter	Exit status, violations, peak memory, duration	cgroup <code>memory.peak</code>

Repository layout

```

zaun/
  src/cli/          main, argument parsing, doctor
  src/policy/       parse (toml++), validate, compile
  src/sandbox/      launcher, init, fs, landlock, seccomp, caps
  src/cgroup/       cgroup v2 manager, rlimit fallback
  src/learn/        notify supervisor, trace log, generalizer
  src/report/       JSON report
  tests/unit/       GoogleTest
  tests/escape/     one program per attack + expected outcome
  bench/           hyperfine scripts and workloads
  integrations/py/  zaun_exec wrapper + agent demo
  docs/            threat-model.md, results.md

```

Dependencies: CMake, libseccomp, toml++ (header-only), nlohmann/json, GoogleTest. Landlock uses raw syscalls from `<linux/landlock.h>` , since glibc has no wrappers.

Detailed design

The order of setup is the core of the design: every restriction must be in place before `execve`, and `seccomp` goes last because it would otherwise block the setup calls.

Launch sequence

```
sequenceDiagram
    participant S as Supervisor
    participant I as Init (PID 1)
    participant T as Target
    S->>I: clone(NEWUSER, NEWNS, NEWPID,<br/>NEWNET, NEWIPC, NEWUTS)
    S->>S: write setgroups=deny, uid_map, gid_map
    S->>S: move child into zaun-<pid> cgroup
    S->>I: release sync pipe
    I->>I: mount / rprivate, build tmpfs root
    I->>I: bind mounts, pivot_root, new /proc
    I->>T: fork
    T->>T: setsid, close fds, scrub env
    T->>T: drop caps, PR_SET_NO_NEW_PRIVS
    T->>T: Landlock restrict, then seccomp
    T->>T: execve(command)
    I->>S: target exit code
```

Init stays as PID 1 so orphans are reaped and signals from the supervisor reach the target, the same job `tini` does in containers.

Namespaces and identity

- Map the caller's uid and gid to themselves inside the user namespace, so files keep correct ownership. Write `deny` to `setgroups` first, as the kernel requires for unprivileged maps.
- The child holds full capabilities inside its own user namespace only long enough to mount. The target clears the effective, permitted, inheritable, ambient and bounding sets before `execve`.
- Ubuntu 23.10 and later restrict unprivileged user namespaces through AppArmor. `zaun doctor` detects this; the fix is a one-time AppArmor profile for the `zaun` binary, or the `sysctl` in development and CI.

Filesystem

- The new root is a size-limited tmpfs. Bind read-only: `/usr` , `/lib` , `/lib64` , `/bin` , `/etc/ld.so.cache` , `/etc/ssl/certs` , plus paths the policy grants. Remount each bind with `MS_RDONLY` | `MS_NOSUID` | `MS_NODEV` , which takes a second `mount` call.
- Bind the workdir read-write. Give the target a fresh tmpfs `/tmp` , a generated minimal `/etc/passwd` , and a `/dev` holding only `null` , `zero` , `full` , `random` , `urandom` . Do not mount `/sys` or `$HOME` .
- Landlock adds a second layer on top of the mounts: read, write, execute and `REFER` (cross-directory rename or link) rules per directory. If a mount rule is ever wrong, Landlock still denies. Detect the Landlock ABI at runtime and degrade with a warning.

Seccomp

Zaun builds the filter with libseccomp, sets the bad-architecture action to kill the process, and turns on its binary-tree optimization. Two layers:

1. **Always-deny list** (cannot be learned away): `io_uring_*` (its operations bypass syscall filtering), `bpf` , `perf_event_open` , `userfaultfd` , `ptrace` , `process_vm_readv/writev` , `keyctl` , `add_key` , `request_key` , `kexec_*` , module loading, `mount` and the new mount API, `pivot_root` , `unshare` , `setns` , and `clone` with any namespace flag.
2. **Learned allowlist** (optional, `mode = "allowlist"`): only syscalls seen during learning, plus a small base set (`exit_group` , `rt_sigreturn` , `futex` , clock calls).

Two details matter. `clone3` passes its flags in a struct pointer that seccomp cannot inspect, so Zaun returns `ENOSYS` for it; glibc then falls back to `clone` , whose flags can be filtered. `ioctl` requests are plain integers, so `TIOCSTI` and `TIOCLINUX` are denied by value. Denied calls return `EPERM` ; the always-deny list kills the process.

Resource limits

- Create `zaun-<pid>` inside the user's delegated cgroup (systemd user sessions delegate `user@<uid>.service`). Set `memory.max` , `memory.swap.max = 0` , `pids.max` , `cpu.max` .
- The supervisor enforces the wall-clock timeout by writing `1` to `cgroup.kill` , which kills every process in the tree at once.

- With no cgroup delegation, fall back to `setrlimit ( RLIMIT_AS , RLIMIT_CPU , RLIMIT_NPROC )` and print that guarantees are weaker. `RLIMIT_NPROC` counts all of the user's processes, not just the sandbox's.

Network

The MVP has one mode: an empty network namespace with only a loopback interface. Abstract Unix sockets are scoped to the network namespace, so this also cuts off D-Bus and X11. Allowlisted egress is a stretch goal (see Risks).

Environment, file descriptors, terminal

- Clear the environment; keep `PATH`, `LANG`, `TERM`, and set `HOME` to the workdir. The policy can add names.
- Call `close_range(3, ~0U, 0)` so no inherited descriptor leaks in.
- Call `setsid()` so the target has no controlling terminal. An interactive `--tty` mode would proxy a new pseudo-terminal (stretch).

Learn mode

Learning must not be a free pass. It uses a wider view so the program behaves normally, but keeps the hard walls:

- The host root is bound read-only, but tmpfs covers a sensitive list: `~/.ssh`, `~/.aws`, `~/.gnupg`, `~/.config/gh`, browser profiles, and credential files.
- The always-deny seccomp list, network isolation and resource limits all stay on.
- The target installs a filter that returns `SECCOMP_RET_USER_NOTIF` for every syscall and sends the listener fd to the supervisor over a socketpair (`SCM_RIGHTS`).
- For each notification, the supervisor reads path arguments with `process_vm_readv`, checks `SECCOMP_IOCTL_NOTIF_ID_VALID`, logs the event, and replies with `SECCOMP_USER_NOTIF_FLAG_CONTINUE`.

The kernel documentation warns that `CONTINUE` must never be used for security decisions: pointer arguments can change after the supervisor reads them (a TOCTOU race). Zaun uses notifications only to observe. Enforcement of paths belongs to mounts and Landlock, which check the path the kernel actually resolves.

Policy generalization

1. Normalize every observed path against the sandbox view; drop `/proc/self` and `fd` paths.
2. Classify each access as read, write/create, or execute (from `execve`).
3. Collapse files under package-managed prefixes (`/usr` , `/lib` , `/etc/ssl`) into directory rules when 3 or more files share a directory.
4. Never collapse into `/` , `$HOME` , `/etc` , or `/proc/sys` . Accesses there become single-file rules plus a warning.
5. Map temporary files to the `/tmp` tmpfs and `workdir` files to the `workdir` rule.
6. Syscalls = union of observed calls + the base set – the always-deny list.
7. Merge several runs by union, and record support per rule ("seen in 3/3 runs") so reviewers can spot one-off accesses.

Kernel requirements

The target is Linux 6.8 (Ubuntu 24.04). Verify these minimums against kernel docs in week 0.

Feature	Minimum kernel
Landlock ABI 4 (TCP bind/connect rules)	6.7
Landlock ABI 3 (truncate)	6.2
Landlock ABI 2 (<code>REFER</code>)	5.19
<code>cgroup.kill</code>	5.14
Landlock ABI 1	5.13
<code>close_range</code>	5.9
<code>SECCOMP_USER_NOTIF_FLAG_CONTINUE</code>	5.5

CLI and policy interface

The CLI has five commands, and every policy is a short TOML file a person can read in under a minute.

Command	What it does
<code>zaun run [--policy p.toml] [--mem 512M] [--timeout 60s] [--workdir DIR] [--report r.json] -- CMD...</code>	Run a command under a policy, or under the default baseline if none is given
<code>zaun learn --out p.toml [--runs N] -- CMD...</code>	Trace the command N times and write a generalized policy
<code>zaun check p.toml</code>	Lint a policy; warn on broad grants such as <code>/</code> , <code>\$HOME</code> , <code>/etc</code>
<code>zaun diff a.toml b.toml</code>	Show added and removed permissions between two policies
<code>zaun doctor</code>	Report kernel version, Landlock ABI, user-namespace and AppArmor status, cgroup delegation

Example policy

```

version = 1
name = "csv-report"

[workdir]
path = "."
mode = "rw"

[filesystem]
read = ["/usr", "/etc/ssl/certs", "/etc/ld.so.cache"]
write = []
exec = ["/usr/bin/python3.12"]

[network]
mode = "none"

[env]
keep = ["PATH", "LANG"]

[limits]
memory = "512M"
pids = 128

```

```

cpu = 1.0
timeout = "60s"

[syscalls]
mode = "allowlist"    # or "baseline"
allow = ["read", "write", "openat", "mmap", "..."]

[meta]
learned_from = 3      # runs

```

Exit codes

Code	Meaning
Child's own code	Target exited normally
125	Zaun failed before <code>execve</code> (bad policy, missing kernel feature)
137	Killed by a limit: memory, pids or timeout (the report says which)
159	Killed by seccomp for an always-deny syscall (128 + <code>SIGSYS</code>)

Report

`--report` writes JSON with the exit reason, duration, peak memory from `memory.peak`, and a violations list. Seccomp denials in allowlist mode are captured by notifying only on denied calls, which costs nothing on the fast path. Landlock denials surface to the program as `EACCES`; `zaun learn` on the same command shows what was missing.

Python integration

```

from zaun import zaun_exec

result = zaun_exec(code, policy="policies/csv-report.toml", timeout=60)
result.stdout, result.exit_reason, result.violations

```

The wrapper writes the code to a temporary workdir, shells out to `zaun run`, and parses the report. It is the piece an agent's code-execution tool calls.

Testing and evaluation

Zaun is judged on three numbers: escape tests blocked, overhead added, and how often a learned policy survives new code. Each goes in the README as a table.

Escape suite

Each test is a small C or Python program in `tests/escape/` with the expected outcome in a sidecar file. A test passes when the attack fails for the expected reason.

ID	Attack	Expected defense
FS-1	Read <code>~/.ssh/id_ed25519</code>	Not mounted
FS-2	Read <code>/etc/shadow</code>	Not mounted
FS-3	Append to <code>~/.bashrc</code>	Not mounted
FS-4	Symlink in workdir pointing into <code>\$HOME</code>	Target absent; Landlock
FS-5	Rename or hardlink a file out of the workdir	Landlock <code>REFER</code>
FS-6	Escape via <code>/proc/1/root</code>	PID namespace
FS-7	Remount <code>/</code> read-write	Caps dropped; seccomp
FS-8	Fill <code>/tmp</code>	tmpfs size limit
NET-1	TCP connect to 1.1.1.1:443	Empty network namespace
NET-2	DNS lookup	No route, no resolver
NET-3	Connect to an abstract Unix socket (D-Bus)	Network namespace scoping
NET-4	Connect to a socket file in host <code>/tmp</code>	Fresh tmpfs <code>/tmp</code>
PROC-1	<code>ptrace</code> the supervisor	PID namespace; seccomp
PROC-2	<code>kill(-1, SIGKILL)</code>	PID namespace
PROC-3	Read another process's <code>/proc/<pid>/environ</code>	PID namespace
PRIV-1	Run a setuid binary	<code>NO_NEW_PRIVS</code> ; <code>nosuid</code>
PRIV-2	<code>unshare(CLONE_NEWUSER)</code>	Seccomp always-deny
PRIV-3	<code>clone3</code> with namespace flags	<code>clone3</code> returns <code>ENOSYS</code>

ID	Attack	Expected defense
PRIV-4	<code>openat</code> through <code>io_uring</code>	<code>io_uring_setup</code> denied
PRIV-5	<code>bpf</code> , <code>perf_event_open</code> , <code>userfaultfd</code>	Seccomp always-deny
PRIV-6	32-bit syscall via <code>int 0x80</code>	Bad-arch action kills
PRIV-7	Read the session keyring with <code>keyctl</code>	Seccomp always-deny
TTY-1	Inject keystrokes with <code>TIOCSTI</code>	<code>setsid</code> ; <code>ioctl</code> filter
RES-1	Fork bomb	<code>pids.max</code>
RES-2	Allocate 8 GB	<code>memory.max</code> OOM kill
RES-3	Infinite loop	Timeout + <code>cgroup.kill</code>
LEAK-1	Read <code>ANTHROPIC_API_KEY</code> from env	Environment scrubbed
LEAK-2	Read an inherited fd 3	<code>close_range</code>

Run each test twice: once bare (the attack must succeed, proving the test is real) and once under Zaun (it must fail). A test that cannot succeed bare is a broken test.

Benchmarks

Time everything with hyperfine, 200 runs, reporting p50 and p95.

Benchmark	Measures	Compared against	Target
<code>/bin/true</code>	Startup latency	Bare, bubblewrap, <code>docker run</code>	< 25 ms over bare
1M <code>getppid</code> / open-close loop	Seccomp filter cost	Bare; baseline vs allowlist filter	< 5%
Python pandas CSV script	Realistic workload	Bare, bubblewrap	< 5%
<code>zaun learn</code> on the same script	Tracing cost	<code>zaun run</code>	Report only

Policy generalization

This is the result that makes Zaun a project and not a wrapper:

1. Pick 20 agent-style tasks (CSV analysis, text processing, running a test suite, image resizing, file compression).
2. For each task, have an LLM generate code once, run `zaun learn`, and review the policy.
3. Generate 5 new samples of code for the same task and run each under the learned policy.
4. Report the success rate, the violations that caused failures, and the average rule count per policy.

CI

- GitHub Actions on `ubuntu-24.04`: set `kernel.apparmor_restrict_unprivileged_userns=0` with `sudo sysctl` before tests.
- Jobs: unit tests, escape suite, a debug build with ASan and UBSan, and clang-tidy.
- Resource tests need cgroup delegation. If the runner lacks it, mark those tests skipped with a clear reason and run them in the VM before each release.
- Upload the escape-suite results table as a build artifact.

Risks, open questions and alternatives

The biggest risks are environmental (distro restrictions, cgroup delegation) and scope creep, not the core sandbox.

Risks

Risk	Impact	Mitigation
AppArmor blocks unprivileged user namespaces on Ubuntu	Zaun cannot start	<code>zaun doctor</code> detects it; ship an AppArmor profile; document the <code>sysctl</code>
No cgroup delegation on the host or CI	No memory or pid limits	<code>setrlimit</code> fallback with a warning; run resource tests in the VM

Risk	Impact	Mitigation
Learned policies overfit to one run	Reruns fail; users widen policies by hand	Learn from several runs; prefix collapsing; measure rerun success
Hostile code poisons learning	Policy grants more than intended	Learning keeps hard walls on; <code>zaun check</code> warns; human review required
Landlock ABI varies by kernel	Weaker filesystem layer on old kernels	Detect ABI at runtime; report in <code>doctor</code> ; mounts remain the first layer
Network allowlist grows into a proxy project	Misses the MVP date	MVP ships network <code>none</code> only
Season and classes compress weeks	Slip past Nov 1	Week 5 is buffer; cut <code>diff</code> and Python wrapper polish first

Alternatives considered

Option	Why not (for this project)
Configure bubblewrap or nsjail	They enforce well but do not learn or review policies; Zaun benchmarks against bubblewrap instead
gVisor or Firecracker	Stronger isolation from kernel bugs, but heavier; the right choice when kernel exploits are in scope
Docker	Needs a daemon and images; startup is far slower than a namespace launch
<code>ptrace</code> tracer for learn mode	Slower, and breaks programs that use <code>ptrace</code> themselves
Rust instead of C++	Memory safety is a real argument for a security tool. C++ ships faster for you; keep the trusted code small, fuzz the policy parser, and run ASan in CI. Expect this question in interviews.

Open questions

- ☐ Workdir default: bind the real directory read-write, or mount an overlay copy and show the diff before applying changes? Overlayfs works inside user namespaces on 5.11+.
- ☐ Policy format: TOML (readable) or JSON (reuses nlohmann/json from Remindr)?
- ☐ Network allowlist design for later: an HTTP CONNECT proxy over a Unix socket (host names), or Landlock TCP rules (ports only)?
- ☐ Architecture: if your Mac is Apple Silicon, the dev VM is aarch64 while CI is x86_64. libseccomp resolves syscall names per architecture, but PRIV-6 is x86-only.

Build plan

The MVP takes six weeks at about 10–12 hours a week: a setup week, four build weeks, and a week for results and write-up. Each week ends with a working binary and a pass/fail exit check, so a bad week slips scope, not the date.

Week	Dates	Milestone	Exit criteria
0	Sep 23–27	Environment and spike	Linux VM running; 100-line spike enters user + mount namespaces, <code>pivot_root</code> s and runs <code>/bin/sh</code> ; <code>zaun doctor</code> prints kernel, Landlock ABI, userns and cgroup status
1	Sep 28–Oct 4	Launcher and filesystem isolation	<code>zaun run -- python3 script.py</code> works; <code>\$HOME</code> invisible; FS-1–4, FS-6, PROC-1–3, LEAK-1–2 pass; CI builds
2	Oct 5–11	Lockdown: caps, Landlock, seccomp, policy files	TOML loader and compiler with unit tests; always-deny list live; FS-5, FS-7, PRIV-1–7, TTY-1 pass
3	Oct 12–18	Limits, network, learn-mode tracer	cgroup limits and timeout; network <code>none</code> ; RES and NET tests pass; raw trace captured for a pandas script

Week	Dates	Milestone	Exit criteria
4	Oct 19–25	Generalizer, tooling, agent demo	<code>learn</code> writes a readable policy; <code>check</code> and violation report work; Python <code>zaun_exec</code> runs an agent over the 20 tasks
5	Oct 26–Nov 1	Results, write-up, buffer	Full escape table, benchmarks, generalization numbers; README with threat model and limitations; demo recording; resume bullet filled

Why this order

- Filesystem isolation comes first because it blocks the most valuable attacks (secrets on disk) and makes every later test meaningful.
- Seccomp comes after mounts because it must be installed last in the launch sequence. Building it on a working launcher avoids debugging two things at once.
- The tracer comes before the generalizer so you design generalization rules from real traces, not guesses.
- The agent demo and numbers come last because they depend on everything else, and they are what you will talk about in interviews.

Cut order if time runs short

1. `zaun diff`
2. Syscall allowlist mode (ship the always-deny baseline only)
3. Docker in the benchmark comparison
4. Reduce the generalization study from 20 tasks to 10

Never cut: the escape suite, the threat model, or the overhead numbers.

Task checklist

Tick items off as you go; each week's list matches its exit criteria in the build plan.

Week 0: Environment and spike

- ☐ Set up an Ubuntu 24.04 VM with 4 GB RAM and a shared folder
- ☐ Install toolchain: clang, CMake, libseccomp-dev, GoogleTest, hyperfine
- ☐ Read `man 7 user_namespaces`, `man 2 seccomp_unotify`, and the kernel Landlock docs; confirm the kernel minimums table
- ☐ Write the throwaway spike: `unshare`, `uid map`, `pivot_root`, `exec /bin/sh`
- ☐ Create the repo, CMake skeleton, and CI workflow with the AppArmor `sysctl`
- ☐ Implement `zaun doctor`

Week 1: Launcher and filesystem

- ☐ `clone` with namespace flags; sync pipe; `uid/gid` maps with `setgroups=deny`
- ☐ `tmpfs` root, read-only bind mounts with the remount step, fresh `/proc`, minimal `/dev`, fresh `/tmp`
- ☐ Init as PID 1: fork target, reap, forward signals, propagate exit code
- ☐ Environment allowlist, `close_range`, `setsid`
- ☐ Escape-test harness (bare run vs Zaun run) and the first 9 tests

Week 2: Lockdown and policies

- ☐ Drop all capability sets; `PR_SET_NO_NEW_PRIVS`
- ☐ Landlock ruleset from policy, with ABI detection and fallback
- ☐ Seccomp always-deny list, bad-arch kill, `clone3` → `ENOSYS`, `ioctl` filter
- ☐ TOML loader with strict validation; policy compiler; unit tests for both
- ☐ Escape tests FS-5, FS-7, PRIV-1-7, TTY-1

Week 3: Limits, network, tracer

- ☐ cgroup v2 manager: create, limit, `cgroup.kill` on timeout, `memory.peak`; `setrlimit` fallback
- ☐ Network `none`; confirm abstract-socket isolation
- ☐ Learn mode: wide read-only view with sensitive paths masked
- ☐ Seccomp notify supervisor: fd passing, argument reads, `NOTIF_ID_VALID`, `CONTINUE`
- ☐ Raw trace log; escape tests RES-1-3, NET-1-4, FS-8

Week 4: Generalizer and agent demo

- ☐ Generalizer rules 1-7 with unit tests on recorded traces
- ☐ Multi-run merge with per-rule support counts
- ☐ `zaun check`, violation report JSON, (optional) `zaun diff`
- ☐ Python `zaun_exec` wrapper
- ☐ Agent script: 20 tasks, 1 learning sample + 5 test samples each

Week 5: Results and write-up

- ☐ Run the full escape suite; publish the results table
- ☐ Run benchmarks against bare, bubblewrap and Docker
- ☐ Run the generalization study; record success rate and failure causes
- ☐ README: quick start, threat model, results, known limitations
- ☐ Terminal recording of learn → review → run → blocked attack
- ☐ Short write-up post on your site

Deliverables

- Public repo with CI badges and a results table in the README
- `docs/threat-model.md` and `docs/results.md`
- Demo recording and write-up post
- Pinned on GitHub with a description that names the hard part

Resume bullet (fill in after week 5)

Built Zaun, a C++17 Linux sandbox for LLM-generated code (namespaces, seccomp-bpf, Landlock, cgroups v2) that learns least-privilege policies from traced runs; blocked [N/N] escape tests at [X] ms startup overhead, and learned policies ran [Y]% of fresh agent-generated programs unchanged.

Prepare answers for

- Why can't seccomp enforce file-path rules, and what does Zaun use instead?
- Why is seccomp installed last, and why return `ENOSYS` for `clone3`?
- What is out of your threat model, and when would you choose gVisor instead?
- How do you stop hostile code from poisoning the learned policy?
- Why C++ and not Rust for a security tool?